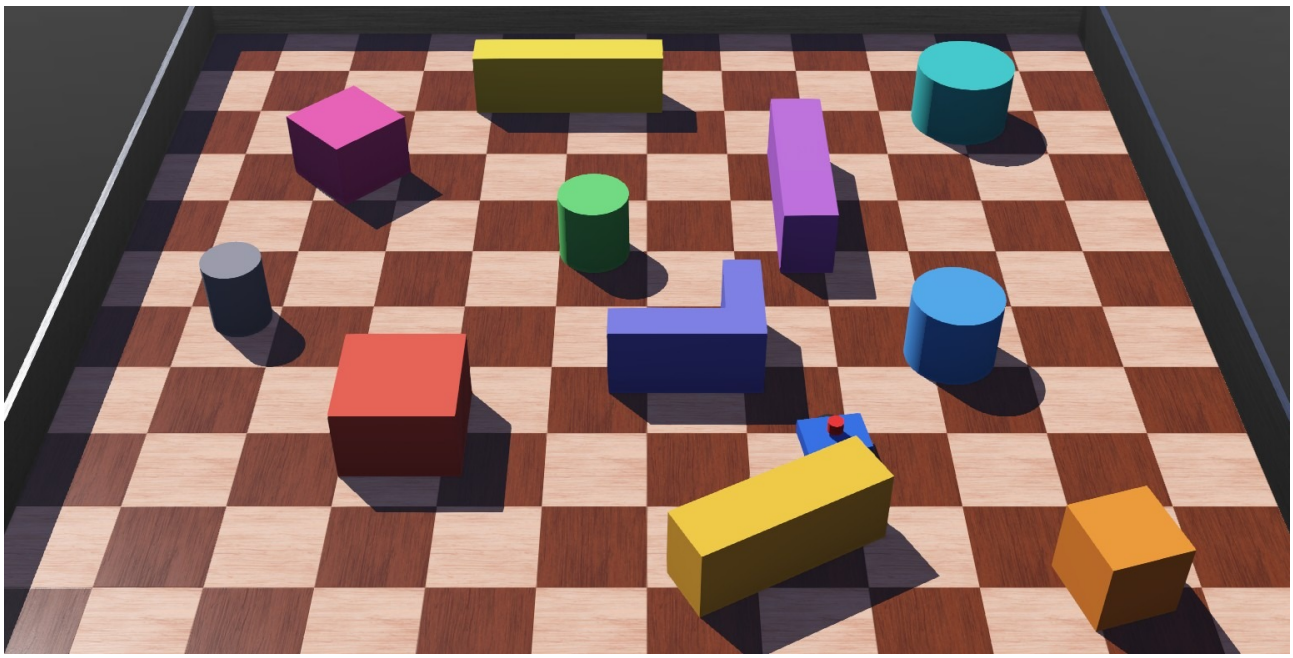
LAPORAN TUGAS PELATIHAN LAB B401

Robot Obstacle Avoidance Berbasis Lidar 360° di Webots

Navigasi reaktif *Corridor Gap Selection* dengan state machine dan lapisan safety ultrasonik



Gambar 1. Arena 3 × 3 m dengan 12 obstacle dan robot differential drive kustom.

Disusun oleh: **Ahnaf**

Repositori: https://github.com/ahnafyura/webots-obstacle-avoidance-b401

Simulator: Cyberbotics Webots R2025a · Bahasa: Python 3

Oktober 2026

1. Pendahuluan

1.1 Latar Belakang

Pada modul pelatihan sebelumnya robot dirakit sebagai *line tracer*: robot differential drive yang mengikuti garis hitam dengan dua sensor inframerah dan kontrol *bang-bang*. Tugas ini mengembangkan robot yang sama menjadi robot **obstacle avoidance**. Robot harus bergerak secara otonom di arena berisi rintangan tanpa menabraknya. Rintangannya tidak hanya dinding pinggir arena, tetapi juga obstacle yang diletakkan di tengah arena.

1.2 Spesifikasi Tugas dan Pemenuhannya

Tabel 1. Pemenuhan ketentuan tugas

Ketentuan tugas	Implementasi pada proyek ini	Status
Membuat robot obstacle avoidance	Robot bergerak otonom dan menghindari semua rintangan. Diuji 8 × 300 detik dengan total 0 tabrakan.	✔
Bentuk robot bebas (boleh sama dengan modul)	Robot kustom dengan struktur seperti modul: body *Box*, 2 roda *HingeJoint* + *RotationalMotor*, dan roda kastor *BallJoint*. Ditambah encoder, IMU, Lidar, dan sonar.	✔
Menaruh obstacle di arena, tidak hanya menghindari pinggiran arena	12 obstacle di tengah arena: kotak, kotak miring, tabung berbagai ukuran, dinding, dan siku-L (perangkap cekung).	✔
Sensor bebas (Lidar, Ultrasonic, dll.)	Lidar 360° (360 sinar, jangkauan 2.5 m) sebagai sensor utama, 3 sensor ultrasonik (sonar) di depan sebagai lapisan safety, serta 2 encoder roda dan IMU untuk odometri.	✔

1.3 Tujuan

- Merancang robot differential drive beserta sensor jarak di Webots.
- Mengimplementasikan algoritma navigasi reaktif yang menghindari obstacle dengan aman dan tetap menjelajahi arena.
- Menguji robot secara kuantitatif, yaitu jumlah tabrakan, jarak aman (clearance), dan cakupan arena, dari beberapa posisi awal.

2. Dasar Teori

2.1 Kinematika Differential Drive

Robot digerakkan oleh dua roda independen. Kecepatan linear v dan kecepatan sudut ω diperoleh dari kecepatan roda kanan V_R dan kiri V_L dengan jarak antar roda L:

$$v = (V_R + V_L) / 2 \quad \omega = (V_R - V_L) / L$$

Controller menghitung v dan ω yang diinginkan, lalu mengubahnya menjadi kecepatan sudut roda (rad/s) dengan jari-jari roda r. Jika salah satu roda melebihi batas motor, kedua nilai diskalakan bersama sehingga arah belok tetap terjaga:

$$\omega_L = (v - \omega \cdot L/2) / r \quad \omega_R = (v + \omega \cdot L/2) / r$$

2.2 Lidar

Lidar (*Light Detection and Ranging*) memancarkan berkas cahaya dan mengukur jarak dari waktu pantulnya. Lidar putar 2D menghasilkan *range image*, yaitu deretan jarak untuk setiap sudut. Pada proyek ini Lidar memiliki 360 sinar dengan resolusi 1° dan FOV 360°. Hasil uji

kalibrasi di Webots: indeks 0 mengarah ke belakang robot (+180°), kemudian urutannya searah jarum jam. Indeks 90 mengarah ke kiri, 180 ke depan, dan 270 ke kanan. Jadi sudut sinar ke-*i* adalah $\theta_i = \pi - i \cdot (2\pi/360)$.

2.3 Sensor Ultrasonik

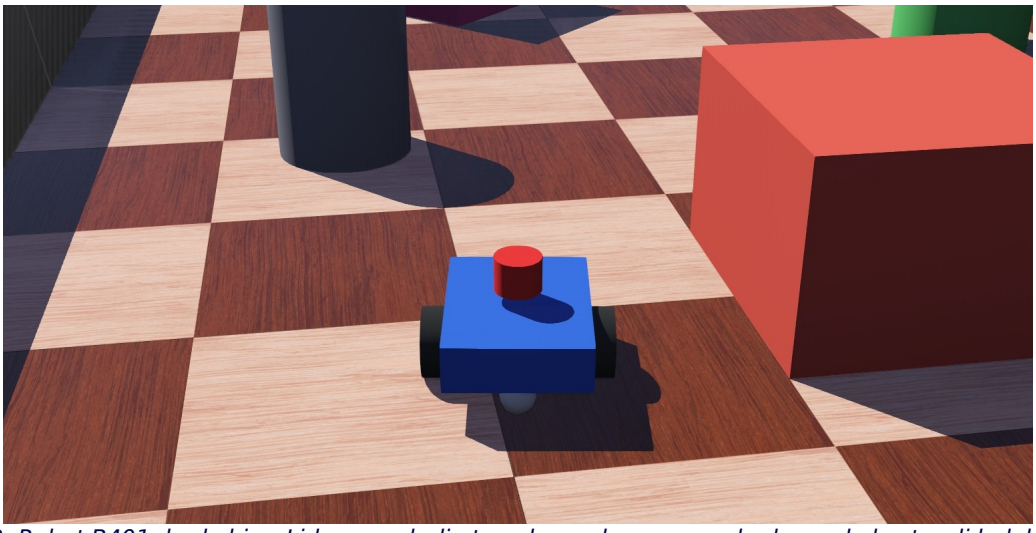
Sensor ultrasonik (sonar) mengukur jarak dari pantulan gelombang suara. Di Webots, sensor ini dimodelkan sebagai *DistanceSensor* bertipe *sonar*. Sensor ini dipakai sebagai lapisan safety kedua di depan robot: jika Lidar gagal melihat obstacle (misalnya obstacle yang lebih rendah dari bidang scan), sonar tetap dapat memicu manuver darurat.

2.4 Navigasi Reaktif

Navigasi reaktif menentukan aksi langsung dari pembacaan sensor saat itu, tanpa peta global. Beberapa pendekatan yang umum adalah *Braitenberg*, *potential field*, *Vector Field Histogram*, dan *Follow-the-Gap*. Proyek ini memakai varian *gap selection* berbasis koridor. Lebar fisik robot diperhitungkan secara eksplisit, sehingga celah yang lebih sempit dari robot tidak akan dipilih.

3. Perancangan Robot

Robot dirakit dari node dasar Webots dengan langkah yang sama seperti modul Line Tracer, yaitu *Robot → Shape (DEF BODY_SHAPE) → HingeJoint* untuk roda dan *BallJoint* untuk kastor. Sensor IR lantai pada modul diganti dengan sensor jarak horizontal.



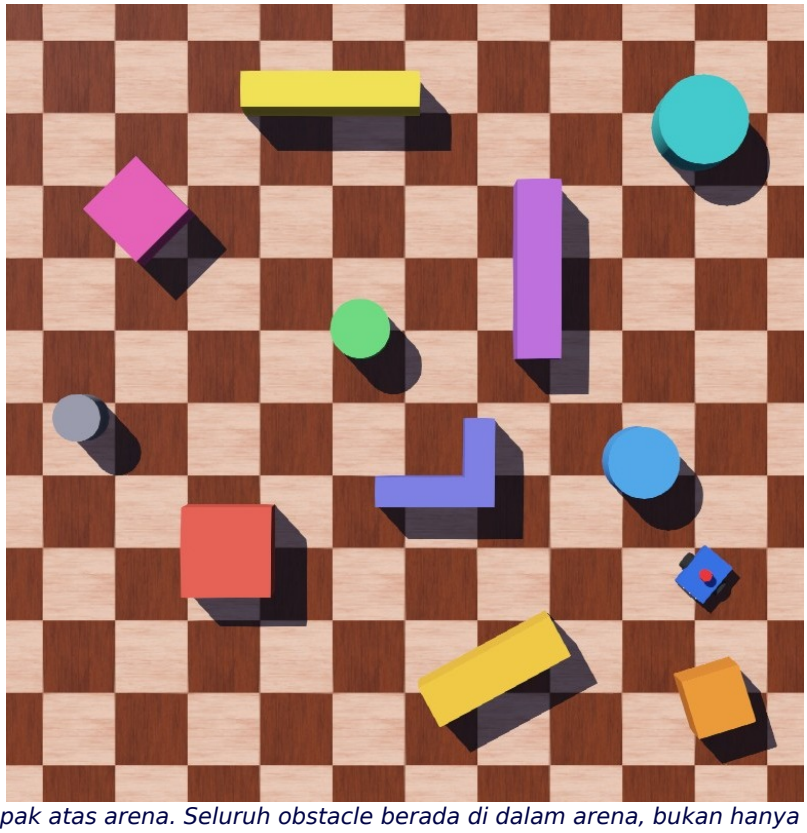
Gambar 2. Robot B401: body biru, Lidar merah di atas, dua roda penggerak, dan roda kastor di belakang. Tiga sensor ultrasonik terpasang di sisi depan.

Tabel 2. Spesifikasi robot

Komponen	Node Webots	Nama device / parameter
Body (chassis)	Shape + Box	0.16 × 0.12 × 0.04 m, massa 0.6 kg, center of mass digeser ke belakang
Roda kiri / kanan	HingeJoint + RotationalMotor	motor_kiri / motor_kanan; r = 0.03 m, L = 0.14 m, v_{maks} 15 rad/s
Encoder roda	PositionSensor	encoder_kiri / encoder_kanan, jarak tempuh untuk odometri
IMU	InertialUnit	imu, arah hadap (yaw) untuk odometri, tidak terpengaruh slip roda
Roda kastor	BallJoint + Sphere	r = 0.015 m, di x = −0.06 m
Lidar 360°	Lidar (cylindrical)	lidar; 360 sinar, 1 layer, 0.03–2.5 m, noise 0.2%, tinggi 9.5 cm
Ultrasonik ×3	DistanceSensor type "sonar"	us_kiri (+34°), us_depan (0°), us_kanan (−34°); keluaran dalam meter (0–1 m)

4. Perancangan Arena

Arena berupa *RectangleArena* 3 × 3 m dengan dinding setinggi 0.2 m. Di dalamnya diletakkan **12 obstacle** dengan bentuk dan orientasi beragam. Tujuannya agar algoritma diuji pada kasus yang sulit: celah sempit, permukaan melengkung (tabung), permukaan miring, dan sudut cekung (siku-L). Sudut cekung adalah kasus klasik yang membuat robot reaktif sederhana terjebak.



Gambar 3. Tampak atas arena. Seluruh obstacle berada di dalam arena, bukan hanya dinding pinggir.

Tabel 3. Daftar obstacle (tinggi semua 0.2 m)

#	Nama	Bentuk	Posisi (x, y)	Ukuran
1	kotak_1	Box	(−0.60, −0.50)	0.30 × 0.30 m
2	dinding_miring	Box, rotasi 29°	(0.30, −0.90)	0.50 × 0.15 m
3	tabung_1	Cylinder	(0.80, −0.20)	r = 0.12 m
4	tabung_2	Cylinder	(−0.15, 0.25)	r = 0.10 m
5	dinding_tengah	Box	(0.45, 0.45)	0.15 × 0.60 m
6	kotak_berlian	Box, rotasi 45°	(−0.90, 0.65)	0.25 × 0.25 m
7	tabung_besar	Cylinder	(1.00, 0.95)	r = 0.15 m
8	dinding_utara	Box	(−0.25, 1.05)	0.60 × 0.12 m
9	tiang	Cylinder	(−1.10, −0.05)	r = 0.08 m
10	kotak_2	Box, rotasi 17°	(1.05, −1.00)	0.20 × 0.20 m
11–12	siku_L_a / siku_L_b	Box (bentuk L)	(0.10, −0.30) / (0.25, −0.15)	0.40 × 0.10 / 0.10 × 0.20 m

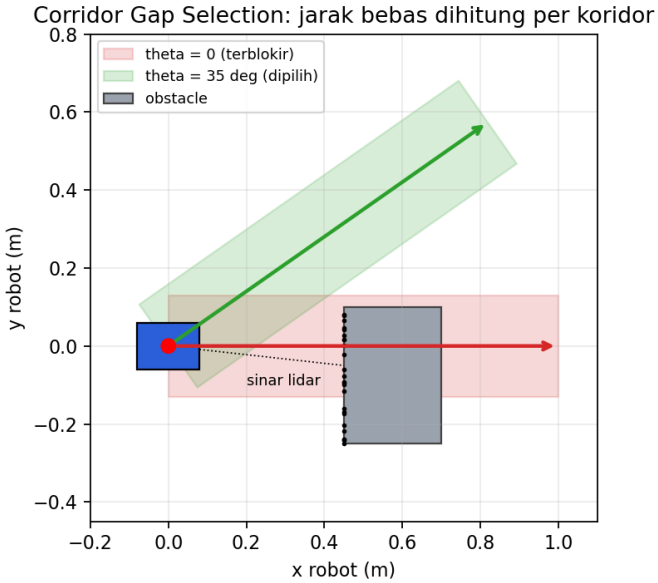
5. Algoritma Navigasi

5.1 Corridor Gap Selection

Setiap langkah simulasi (16 ms), scan Lidar diubah ke titik-titik (x, y) dalam frame robot. Untuk 61 kandidat arah θ dari −150° sampai +150° (kelipatan 5°), controller menghitung **jarak bebas koridor**. Koridor adalah persegi panjang selebar robot ditambah margin (2 × 0.13 m) yang memanjang ke arah θ. Setiap titik Lidar diproyeksikan ke arah θ:

$$\text{longitudinal} = x \cdot \cos\theta + y \cdot \sin\theta \quad \text{lateral} = -x \cdot \sin\theta + y \cdot \cos\theta$$

Titik dengan nilai longitudinal positif dan |lateral| lebih kecil dari setengah lebar koridor berarti memblokir koridor. Jarak bebas adalah nilai longitudinal terkecil dari titik-titik tersebut, dengan batas maksimum 1 m. Cara ini lebih aman daripada hanya melihat satu sinar, karena celah yang terlihat lebar tetapi sebenarnya lebih sempit dari badan robot tidak akan lolos seleksi.



Gambar 4. Konsep koridor. Koridor θ = 0 terblokir obstacle, sedangkan koridor θ = 35° bebas sehingga dipilih.

Setiap kandidat diberi skor sebagai berikut, lalu kandidat dengan skor terbesar dipilih:

$$\text{skor}(\theta) = \min(\text{bebas}(\theta), 1\ \text{m}) - 0.15 \cdot |\theta - \theta_{\text{jelajah}}| - 0.08 \cdot |\theta - \theta_{\text{sebelumnya}}|$$

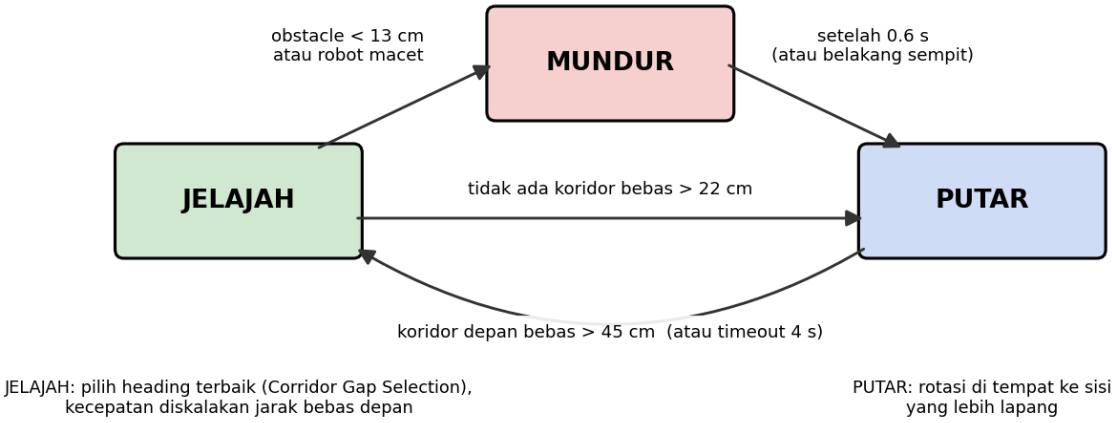
- **Suku pertama** mengutamakan arah yang paling lapang.
- **Suku kedua** (bias jelajah) membuat robot menjelajah. Setiap 5–10 detik robot memilih arah global acak, lalu arah itu dikonversi ke frame robot memakai heading dari odometri encoder. Tanpa suku ini, robot cenderung berputar di satu lintasan yang sama (limit cycle).
- **Suku ketiga** (smoothing) mencegah robot zig-zag karena arah tujuan berubah-ubah.

Kecepatan dihitung dengan kontrol proporsional:

$$\omega = \text{clamp}(2.8 \cdot \theta^*, \pm 3\ \text{rad/s})\quad v = 0.28 \cdot \text{clamp}((d_{\text{depan}} - 0.17)/(0.60 - 0.17), 0, 1) \cdot \cos^2\theta^*$$

Robot melaju penuh di area lapang, melambat ketika mendekati obstacle, dan hampir berputar di tempat ketika arah yang dipilih jauh ke samping.

5.2 State Machine dan Penanganan Kondisi Khusus



Gambar 5. State machine controller.

- **JELAJAH**: navigasi normal dengan Corridor Gap Selection.
- **MUNDUR**: aktif jika titik Lidar terdekat di sektor depan ±60° kurang dari 13 cm, sonar membaca kurang dari 5 cm, atau robot terdeteksi macet. Robot mundur 0.10 m/s sambil memutar ke sisi yang lebih lapang, dengan syarat bagian belakang masih kosong.
- **PUTAR**: berputar di tempat sampai koridor depan bebas lebih dari 45 cm. Jika jarak ke segala arah kurang dari 11.5 cm, robot terlalu sempit untuk berputar, sehingga robot bergeser menjauh dari titik terdekat terlebih dahulu.
- **Watchdog macet**: rata-rata perubahan scan Lidar dibandingkan dengan 2 detik sebelumnya. Jika perubahannya kurang dari 4 mm padahal robot sedang diperintah maju, robot dianggap tersangkut dan masuk state MUNDUR.

5.3 Potongan Kode Inti

```
def corridor_free(self, scan, theta, half_width):
    """Jarak bebas sepanjang koridor selebar 2*half_width ke arah theta."""
    ct, st = math.cos(theta), math.sin(theta)
    free = LOOKAHEAD
    for i, r in enumerate(scan):
        if r >= LOOKAHEAD + half_width:
            continue
        x = r * self.ray_cos[i]
```

```
        y = r * self.ray_sin[i]
        longi = x * ct + y * st          # komponen searah theta
        if longi <= 0.0:
            continue
        lat = -x * st + y * ct           # komponen menyamping
        if abs(lat) < half_width and longi < free:
            free = longi
    return free

# --- pemilihan arah pada state JELAJAH ---
for th in CANDIDATES:
    free = self.corridor_free(scan, th, hw)
    score = (min(free, LOOKAHEAD)
             - K_BIAS * abs(th - self.bias)
             - K_SMOOTH * abs(th - self.prev_theta))
    if score > best_score:
        best, best_score, best_free = th, score, free
```

6. Pengujian dan Hasil

6.1 Metode Pengujian

Supaya hasilnya objektif dan tidak hanya berdasarkan pengamatan visual, world dilengkapi robot **evaluator** (Supervisor). Evaluator hanya mengamati dan tidak memengaruhi robot. Tugas-tugasnya:

- **Deteksi tabrakan**: memakai *contact points* robot dari physics engine. Setiap kontak di atas permukaan lantai (z > 8 mm) dihitung sebagai tabrakan. Metode ini sudah divalidasi dengan controller "buta" yang hanya maju lurus. Controller tersebut tercatat menabrak 68 kali dalam 20 detik.
- **Clearance**: jarak analitik dari pusat robot ke permukaan obstacle atau dinding terdekat, dikurangi radius pembungkus robot (0.10 m). Nilai ini konservatif karena body robot berbentuk persegi panjang dan lebih kecil dari lingkaran pembungkusnya.
- **Coverage**: persentase sel grid 10 × 10 cm yang dapat dilalui robot dan sudah dikunjungi.

Pengujian dijalankan secara otomatis dan headless (`tools/stress_test.sh`) dari 8 posisi dan orientasi awal yang berbeda, masing-masing selama 300 detik simulasi dengan seed acak yang berbeda.

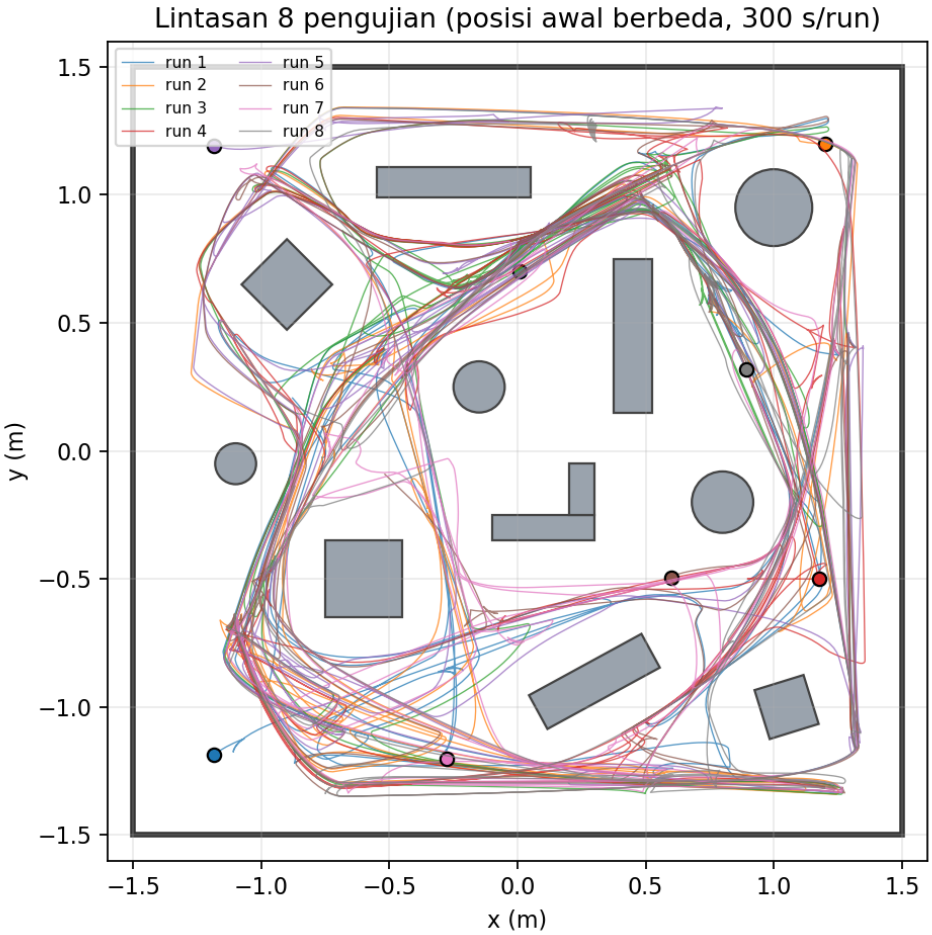
6.2 Hasil

Tabel 4. Hasil uji ketahanan (8 run × 300 detik)

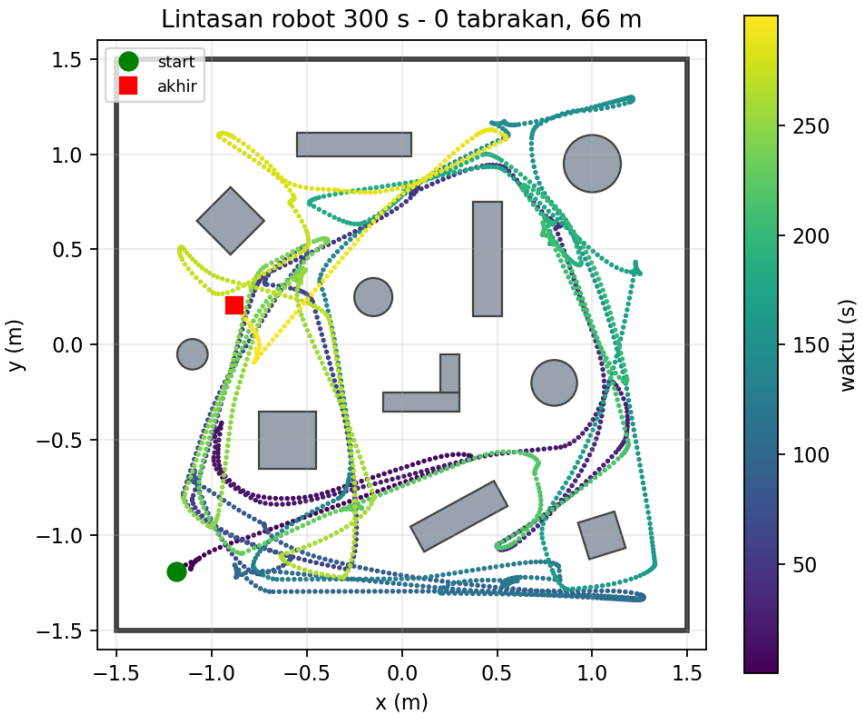
Run	Posisi awal (x, y, heading)	Jarak tempuh	Tabrakan	Clearance min*	Coverage
1	(-1.2, -1.2, 45°)	65.7 m	0	2.8 cm	51.2 %
2	(1.2, 1.2, -132°)	67.2 m	0	2.9 cm	51.9 %
3	(0.0, 0.7, 0°)	64.0 m	0	2.5 cm	51.2 %
4	(1.2, -0.5, 180°)	64.3 m	0	2.4 cm	53.5 %
5	(-1.2, 1.2, -29°)	62.6 m	0	2.4 cm	53.6 %
6	(0.6, -0.5, 90°)	63.5 m	0	2.7 cm	55.9 %
7	(-0.3, -1.2, 0°)	66.1 m	0	3.2 cm	53.8 %
8	(0.9, 0.3, 115°)	63.7 m	0	2.3 cm	50.2 %
Total	**40 menit simulasi**	**517 m**	**0**	**2.3 cm**	**52.7 %** (rata-rata)

*Clearance diukur dari lingkaran pembungkus robot berjari-jari 10 cm, sehingga jarak sebenarnya antara body robot dan obstacle lebih besar dari nilai ini.

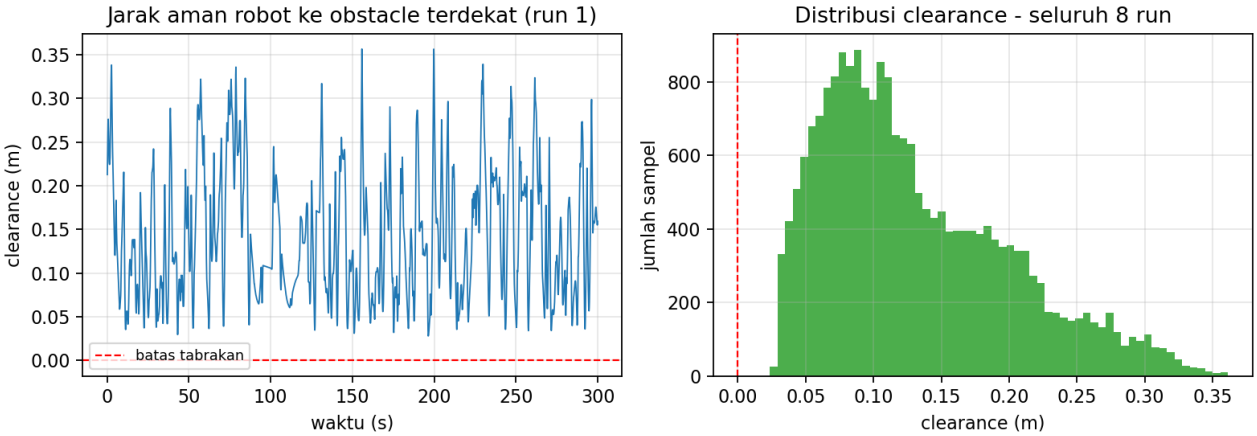
Selama 40 menit simulasi dengan total jarak tempuh 517 m, robot **tidak pernah menabrak** obstacle maupun dinding (0 tabrakan). Nilai clearance terkecil adalah 2.3 cm terhadap lingkaran pembungkus, dan tidak pernah mendekati nol. Dalam 5 menit robot rata-rata menjangkau 53 % area yang dapat dilalui. Robot melewati celah di antara obstacle dan tidak hanya mengelilingi pinggir arena.



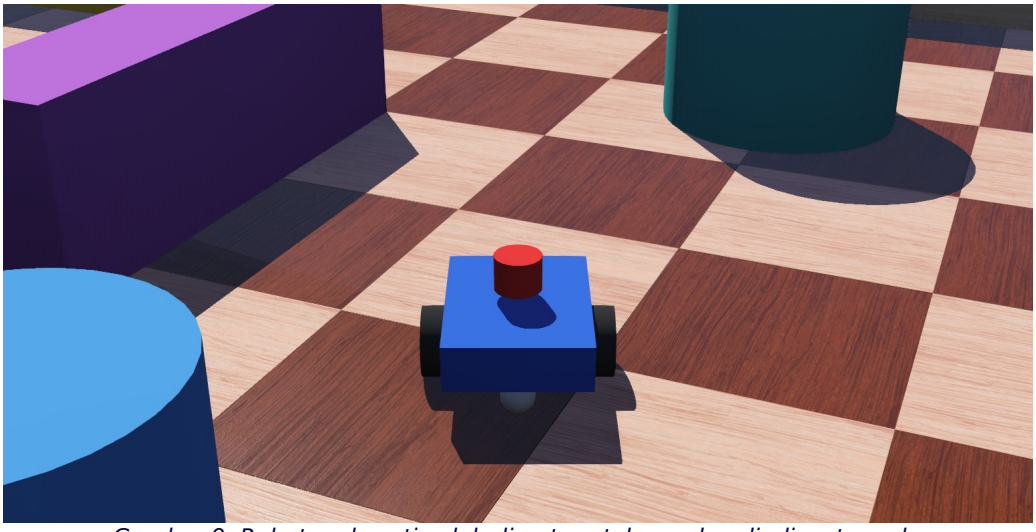
Gambar 6. Lintasan 8 run. Titik bulat menandai posisi awal. Robot menjelajahi hampir seluruh arena dan melewati celah di antara obstacle.



Gambar 7. Lintasan run 1, warna menunjukkan waktu.



Gambar 8. Kiri: clearance terhadap waktu pada run 1. Kanan: distribusi clearance dari seluruh run. Tidak ada sampel di bawah nol.



Gambar 9. Robot melewati celah di antara tabung dan dinding_tengah.

6.3 Evolusi Desain

Versi awal memakai bias jelajah acak dalam frame robot. Robot memang tidak menabrak, tetapi lintasannya menyatu menjadi satu lingkaran besar di sekitar kelompok obstacle tengah, dengan coverage sekitar 38 %. Penyebabnya, bias yang konstan dalam frame robot membuat robot terus berbelok ke arah yang sama. Setelah bias diubah menjadi arah global (memakai odometri), coverage naik menjadi sekitar 53-55 % dan jumlah tabrakan tetap 0.

Perbaikan berikutnya menyangkut odometri. Saat data Lidar ditampilkan di RViz2, titik scan makin bergeser dari posisi obstacle seiring waktu. Penyebabnya adalah drift heading dari encoder, karena roda slip saat robot berputar di tempat. Diukur dari posisi sebenarnya, error median titik scan mencapai 5.5 cm setelah sekitar 15 detik. Setelah heading diambil dari IMU (InertialUnit) dan encoder hanya dipakai untuk jarak tempuh, error turun menjadi 0.4 cm pada detik ke-15 dan 1.7 cm pada detik ke-40. Uji 8 run diulang setelah perubahan ini dan hasilnya tetap 0 tabrakan. Data pada Tabel di atas adalah hasil versi terakhir.

7. Visualisasi di RViz2 (ROS 2)

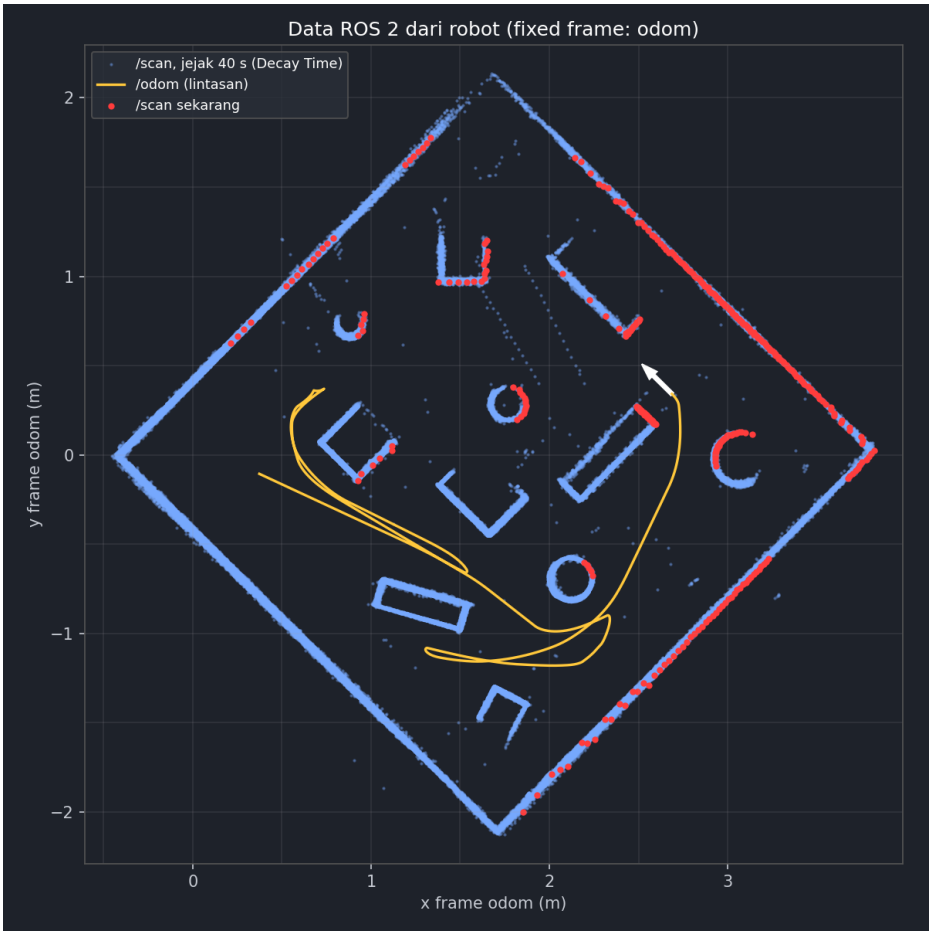
Controller dilengkapi jembatan ROS 2 opsional (`ros_bridge.py`). Jika Webots dijalankan dari terminal yang sudah men-*source* ROS 2 (Jazzy), controller otomatis mempublish data berikut. Jika ROS 2 tidak tersedia, controller tetap berjalan normal tanpa bridge.

Tabel 5. Topic ROS 2 yang dipublish controller

Topic	Tipe pesan	Isi
/scan	sensor_msgs/LaserScan	Scan Lidar 360°. Urutan sinar Webots dibalik agar sudut naik berlawanan jarum jam sesuai konvensi ROS (angle_min = −π + 1°).
/odom	nav_msgs/Odometry	Posisi dan kecepatan robot dari encoder + IMU.
/oa/decision	visualization_msgs/MarkerArray	Koridor dan arah yang sedang dipilih, warna sesuai state.
/tf, /tf_static	tf2	odom → base_link → lidar (tinggi 6.5 cm).

Konvensi LaserScan diverifikasi secara numerik: titik scan ditransformasikan ke koordinat world lalu diukur jaraknya ke permukaan obstacle terdekat. Dengan konvensi bridge, error median 0.6 cm. Jika urutan sudut dibalik, error median menjadi 17 cm. Perintah `tools/rviz.sh` menjalankan Webots dan RViz2 sekaligus dengan konfigurasi siap pakai

(`rviz/obstacle_avoidance.rviz`).



Gambar 10. Data /scan (jejak 40 detik) dan /odom yang direkam dari robot dalam frame odom. Dinding arena dan obstacle terbentuk jelas. Arena tampak miring 45° karena frame odom mengikuti arah hadap awal robot.

8. Cara Menjalankan

- Instal Webots R2025a (atau versi yang lebih baru) dan Python 3.
- Clone repositori: `git clone https://github.com/ahnafyura/webots-obstacle-avoidance-b401.git`
- Di Webots, buka `File → Open World…` lalu pilih `worlds/obstacle_avoidance.wbt`, kemudian tekan Play.
- Console menampilkan state, jarak bebas, dan kecepatan setiap detik. Evaluator mencetak jumlah tabrakan, clearance, dan coverage setiap 10 detik.
- Aktifkan `View → Optional Rendering → Show Lidar Ray Paths` untuk melihat sinar Lidar.
- Tampilan RViz2 (butuh ROS 2 Jazzy): `tools/rviz.sh`.
- Uji otomatis headless: `tools/run_test.sh 180` (satu run) atau `tools/stress_test.sh 300` (8 run). Grafik dibuat ulang dengan `python3 tools/plot_results.py`.

Struktur repositori:

```
webots-obstacle-avoidance-b401/
├── worlds/obstacle_avoidance.wbt          # arena + 12 obstacle + robot + evaluator
├── controllers/obstacle_avoidance/        # controller robot (Lidar + sonar)
├── controllers/evaluator/                 # supervisor pengukur tabrakan/clearance/coverage
├── tools/run_test.sh, stress_test.sh      # pengujian otomatis headless
├── tools/plot_results.py, make_report.py  # grafik & laporan PDF
└── docs/                                  # laporan PDF, gambar, hasil uji
```

9. Kesimpulan

- Robot differential drive kustom dengan Lidar 360°, tiga sensor ultrasonik, encoder, dan IMU berhasil dibuat di Webots dengan struktur node yang sama seperti modul pelatihan.
- Algoritma Corridor Gap Selection memperhitungkan lebar robot secara eksplisit, sehingga robot hanya memilih celah yang benar-benar dapat dilalui. Ditambah state machine (JELAJAH, MUNDUR, PUTAR) dan watchdog macet, robot dapat keluar dari sudut cekung.
- Pengujian kuantitatif selama 40 menit simulasi dari 8 posisi awal menghasilkan **0 tabrakan** dengan total jarak tempuh 517 m dan coverage rata-rata 53 % dalam 5 menit.
- Data Lidar dan odometri (encoder + IMU) dapat dipantau langsung di RViz2 melalui jembatan ROS 2, dengan titik scan yang tepat di permukaan obstacle.
- Pengembangan selanjutnya: SLAM/occupancy grid untuk eksplorasi yang lebih sistematis, navigasi menuju titik tujuan (goal seeking), atau obstacle bergerak.

Lampiran A. Kode Lengkap Controller obstacle_avoidance.py

```
"""
Controller : obstacle_avoidance
Tugas      : Pelatihan Lab B401 - Robot Obstacle Avoidance (Webots, Python)

Robot      : differential drive kustom (body Box, 2 roda HingeJoint, 1 roda kastor
             BallJoint) - struktur sama seperti modul Line Tracer.
Sensor     : 1x Lidar 360 derajat (sensor utama)
             3x DistanceSensor tipe sonar / ultrasonik di depan (lapisan safety)
             2x PositionSensor (encoder roda) + 1x InertialUnit (IMU) untuk odometri

Algoritma  : "Corridor Gap Selection" + state machine
  1. Setiap step, hasil scan Lidar diubah ke koordinat robot (x maju, y kiri).
  2. Untuk sejumlah kandidat arah (heading) theta, dihitung JARAK BEBAS di sepanjang
     koridor selebar robot (+ margin) ke arah theta. Titik lidar yang masuk ke
     dalam koridor dianggap memblokir jalan.
  3. Dipilih arah dengan skor terbaik:
        skor = min(jarak_bebas, LOOKAHEAD)
               - K_BIAS  * |theta - bias_jelajah|
               - K_SMOOTH* |theta - theta_sebelumnya|
     sehingga robot memilih jalur yang lapang, tetap ingin "menjelajah", dan
     tidak zig-zag. bias_jelajah = arah global acak (diganti tiap 5-10 s) yang
     dikonversi ke frame robot memakai odometri encoder.
  4. Kecepatan linear diskalakan dengan jarak bebas di depan (melambat saat dekat
     obstacle), kecepatan sudut proporsional terhadap theta (kontrol P).
  5. State machine:
        JELAJAH : navigasi normal (langkah 2-4)
        MUNDUR  : ada obstacle terlalu dekat di depan -> mundur sambil memutar
        PUTAR   : berputar di tempat menuju arah paling lapang
     Ditambah watchdog "macet": jika scan lidar hampir tidak berubah padahal robot
     diperintah bergerak, robot dianggap tersangkut lalu masuk MUNDUR.
"""

import math
import os
import random

from controller import Robot

# ---------------------------------------------------------------------------
# Parameter fisik robot (harus sesuai dengan world file)
# ---------------------------------------------------------------------------
WHEEL_RADIUS = 0.03        # m
WHEEL_BASE = 0.14          # m, jarak antar roda kiri-kanan
MAX_WHEEL_SPEED = 14.0     # rad/s (motor maxVelocity = 15)

# ---------------------------------------------------------------------------
# Parameter navigasi
# ---------------------------------------------------------------------------
V_MAX = 0.28               # m/s kecepatan maju maksimum
W_MAX = 3.0                # rad/s kecepatan putar maksimum
K_W = 2.8                  # gain proporsional untuk belok

ROBOT_HALF_WIDTH = 0.08    # m (setengah lebar robot termasuk roda)
SIDE_MARGIN = 0.05         # m, margin aman di kiri/kanan koridor
LOOKAHEAD = 1.0            # m, jarak pandang maksimum yang dihitung
D_STOP = 0.17              # m (dari pusat robot) -> v = 0
D_SLOW = 0.60              # m, mulai melambat
D_EMERGENCY = 0.13         # m, jarak titik terdekat di depan -> MUNDUR
D_ROTATE_SAFE = 0.115      # m, minimal clearance agar aman berputar di tempat

K_BIAS = 0.15              # bobot preferensi arah jelajah (per radian)
K_SMOOTH = 0.08            # bobot penalti perubahan arah (per radian)
CANDIDATES = [math.radians(a) for a in range(-150, 151, 5)]

WANDER_MIN = 5.0           # s, interval minimum ganti arah jelajah
WANDER_MAX = 10.0          # s, interval maksimum ganti arah jelajah

STUCK_WINDOW = 2.0         # s
STUCK_SCAN_DIFF = 0.004    # m, rata-rata perubahan scan dianggap "diam"

LOG_PERIOD = 1.0           # s

# Urutan data lidar (FOV 360): indeks 0 = belakang (+FOV/2 = +180 deg), lalu searah
```

```
# jarum jam: 90 = kiri, 180 = depan, 270 = kanan. Diverifikasi dengan uji di world.
LIDAR_FIRST_ANGLE_SIGN = 1.0


def clamp(x, lo, hi):
    return lo if x < lo else hi if x > hi else x


def wrap(a):
    return math.atan2(math.sin(a), math.cos(a))


class ObstacleAvoider:
    def __init__(self):
        self.robot = Robot()
        self.dt_ms = int(self.robot.getBasicTimeStep())
        self.dt = self.dt_ms / 1000.0

        # --- Aktuator ---
        self.motor_l = self.robot.getDevice('motor_kiri')
        self.motor_r = self.robot.getDevice('motor_kanan')
        for m in (self.motor_l, self.motor_r):
            m.setPosition(float('inf'))
            m.setVelocity(0.0)

        # --- Encoder ---
        self.enc_l = self.robot.getDevice('encoder_kiri')
        self.enc_r = self.robot.getDevice('encoder_kanan')
        self.enc_l.enable(self.dt_ms)
        self.enc_r.enable(self.dt_ms)

        # --- Lidar ---
        self.lidar = self.robot.getDevice('lidar')
        self.lidar.enable(self.dt_ms)
        # point cloud diperlukan agar 'Show Lidar Point Cloud' di Webots tampil
        self.lidar.enablePointCloud()
        self.n_rays = self.lidar.getHorizontalResolution()
        self.fov = self.lidar.getFov()
        self.max_range = self.lidar.getMaxRange()
        # sudut tiap sinar dalam frame robot (0 = depan, + = kiri)
        step = self.fov / self.n_rays
        self.ray_angles = [LIDAR_FIRST_ANGLE_SIGN * (self.fov / 2.0 - i * step)
                           for i in range(self.n_rays)]
        self.ray_cos = [math.cos(a) for a in self.ray_angles]
        self.ray_sin = [math.sin(a) for a in self.ray_angles]

        # --- IMU: heading lebih akurat dari encoder (tidak terpengaruh slip roda) ---
        self.imu = self.robot.getDevice('imu')
        self.imu.enable(self.dt_ms)
        self.imu_yaw0 = None

        # --- Ultrasonik (safety layer) ---
        self.us = {}
        for name in ('us_kiri', 'us_depan', 'us_kanan'):
            s = self.robot.getDevice(name)
            s.enable(self.dt_ms)
            self.us[name] = s

        # --- State ---
        self.state = 'JELAJAH'
        self.state_timer = 0.0
        self.turn_dir = 1.0
        self.prev_theta = 0.0
        self.bias = 0.0
        self.wander_heading = 0.0        # arah jelajah global (frame odometri)
        self.wander_timer = 0.0
        self.odom_yaw = 0.0              # heading hasil odometri encoder
        self.odom_x = 0.0                # posisi hasil odometri (frame odom)
        self.odom_y = 0.0
        self.chosen = (0.0, 0.0)         # (theta, jarak_bebas) terakhir, untuk RViz
        self.prev_enc = None
        self.log_timer = 0.0
        self.scan_history = []           # (waktu, scan) untuk deteksi macet
        self.cmd_v = 0.0
        self.time = 0.0
        self.stats = {'mundur': 0, 'macet': 0}
        random.seed(int(os.environ.get('OA_SEED', '401')))
```

```
        # --- ROS 2 bridge (opsional, untuk RViz2) ---
        self.ros = None
        if os.environ.get('OA_ROS', '1') != '0':
            try:
                from ros_bridge import RosBridge
                self.ros = RosBridge(0.065, self.n_rays, self.fov,
                                     self.lidar.getMinRange(), self.max_range)
            except ImportError:
                pass                    # ROS 2 tidak di-source: jalan tanpa bridge

    # ------------------------------------------------------------------
    # Pengolahan sensor
    # ------------------------------------------------------------------
    def read_scan(self):
        raw = self.lidar.getRangeImage()
        scan = []
        for r in raw:
            if r != r or r == float('inf') or r <= 0.0 or r > self.max_range:
                r = self.max_range
            scan.append(r)
        return scan

    def corridor_free(self, scan, theta, half_width):
        """Jarak bebas sepanjang koridor selebar 2*half_width ke arah theta."""
        ct, st = math.cos(theta), math.sin(theta)
        free = LOOKAHEAD
        for i, r in enumerate(scan):
            if r >= LOOKAHEAD + half_width:
                continue
            x = r * self.ray_cos[i]
            y = r * self.ray_sin[i]
            longi = x * ct + y * st          # komponen searah theta
            if longi <= 0.0:
                continue
            lat = -x * st + y * ct           # komponen menyamping
            if abs(lat) < half_width and longi < free:
                free = longi
        return free

    def sector_min(self, scan, a_min, a_max):
        m = self.max_range
        for i, r in enumerate(scan):
            a = self.ray_angles[i]
            if a_min <= a <= a_max and r < m:
                m = r
        return m

    def is_stuck(self, scan):
        self.scan_history.append((self.time, scan))
        while self.scan_history and self.time - self.scan_history[0][0] > STUCK_WINDOW:
            self.scan_history.pop(0)
        if self.time < STUCK_WINDOW + 0.5 or abs(self.cmd_v) < 0.05:
            return False
        old = self.scan_history[0][1]
        diff = sum(abs(a - b) for a, b in zip(scan, old)) / len(scan)
        return diff < STUCK_SCAN_DIFF

    # ------------------------------------------------------------------
    # Penggerak
    # ------------------------------------------------------------------
    def drive(self, v, w):
        wl = (v - w * WHEEL_BASE / 2.0) / WHEEL_RADIUS
        wr = (v + w * WHEEL_BASE / 2.0) / WHEEL_RADIUS
        scale = max(1.0, abs(wl) / MAX_WHEEL_SPEED, abs(wr) / MAX_WHEEL_SPEED)
        self.motor_l.setVelocity(wl / scale)
        self.motor_r.setVelocity(wr / scale)
        self.cmd_v = v

    def set_state(self, s):
        if s != self.state:
            self.state = s
            self.state_timer = 0.0

    # ------------------------------------------------------------------
    # Loop utama
    # ------------------------------------------------------------------
    def step(self):
        scan = self.read_scan()
```

```
us_min = min(s.getValue() for s in self.us.values())

hw = ROBOT_HALF_WIDTH + SIDE_MARGIN
front_free = self.corridor_free(scan, 0.0, ROBOT_HALF_WIDTH + 0.02)
front_min = self.sector_min(scan, -math.radians(60), math.radians(60))
rear_min = self.sector_min(scan, math.radians(120), math.pi)
rear_min = min(rear_min, self.sector_min(scan, -math.pi, -math.radians(120)))
all_min = min(scan)
left_space = sum(scan[i] for i, a in enumerate(self.ray_angles) if 0.2 < a < 1.8)
right_space = sum(scan[i] for i, a in enumerate(self.ray_angles) if -1.8 < a < -0.2)

# odometri heading dari encoder roda (dead reckoning)
enc = (self.enc_l.getValue(), self.enc_r.getValue())
if self.prev_enc is not None:
    dl = (enc[0] - self.prev_enc[0]) * WHEEL_RADIUS
    dr = (enc[1] - self.prev_enc[1]) * WHEEL_RADIUS
    ds = (dr + dl) / 2.0
    imu_yaw = self.imu.getRollPitchYaw()[2]
    if imu_yaw == imu_yaw:                       # bukan NaN
        if self.imu_yaw0 is None:
            self.imu_yaw0 = imu_yaw
        new_yaw = wrap(imu_yaw - self.imu_yaw0)  # heading dari IMU
    else:
        new_yaw = wrap(self.odom_yaw + (dr - dl) / WHEEL_BASE)
    mid = self.odom_yaw + wrap(new_yaw - self.odom_yaw) / 2.0
    self.odom_x += ds * math.cos(mid)
    self.odom_y += ds * math.sin(mid)
    self.odom_yaw = new_yaw
self.prev_enc = enc

# arah jelajah: robot memilih arah global acak setiap beberapa detik,
# sehingga bergerak lurus-lurus ke berbagai arah dan menjelajahi arena
# (tidak berputar-putar di satu lintasan).
self.wander_timer -= self.dt
if self.wander_timer <= 0.0:
    self.wander_timer = random.uniform(WANDER_MIN, WANDER_MAX)
    self.wander_heading = random.uniform(-math.pi, math.pi)
self.bias = clamp(wrap(self.wander_heading - self.odom_yaw),
                  CANDIDATES[0], CANDIDATES[-1])

emergency = front_min < D_EMERGENCY or us_min < D_EMERGENCY - 0.08
stuck = self.is_stuck(scan)
self.state_timer += self.dt

v = w = 0.0
if self.state == 'JELAJAH':
    if emergency or stuck:
        self.stats['mundur'] += 1
        if stuck:
            self.stats['macet'] += 1
        self.turn_dir = 1.0 if left_space > right_space else -1.0
        self.set_state('MUNDUR')
    else:
        best, best_score, best_free = 0.0, -1e9, 0.0
        for th in CANDIDATES:
            free = self.corridor_free(scan, th, hw)
            score = (min(free, LOOKAHEAD)
                     - K_BIAS * abs(th - self.bias)
                     - K_SMOOTH * abs(th - self.prev_theta))
            if score > best_score:
                best, best_score, best_free = th, score, free
        if best_free < D_STOP + 0.05:
            # tidak ada jalan yang cukup lapang -> putar di tempat
            self.turn_dir = 1.0 if left_space > right_space else -1.0
            self.set_state('PUTAR')
        else:
            self.prev_theta = best
            self.chosen = (best, best_free)
            w = clamp(K_W * best, -W_MAX, W_MAX)
            speed_factor = clamp((front_free - D_STOP) / (D_SLOW - D_STOP), 0.0, 1.0)
            heading_factor = max(0.0, math.cos(best)) ** 2
            v = V_MAX * speed_factor * heading_factor
            if v < 0.03 and front_free > D_STOP + 0.03:
                v = 0.03
            if front_free <= D_STOP:
                v = 0.0

elif self.state == 'MUNDUR':
```

```
            # mundur pelan sambil memutar menjauhi obstacle
            if rear_min > 0.16:
                v = -0.10
            w = self.turn_dir * 1.5
            if self.state_timer > 0.6 or (rear_min <= 0.16 and self.state_timer > 0.1):
                self.set_state('PUTAR')

        elif self.state == 'PUTAR':
            if all_min < D_ROTATE_SAFE:
                # terlalu sempit untuk berputar: geser menjauh dari titik terdekat
                i_min = scan.index(all_min)
                v = -0.06 if abs(self.ray_angles[i_min]) < math.pi / 2 else 0.06
            w = self.turn_dir * 2.2
            path = self.corridor_free(scan, 0.0, hw)
            if (path > 0.45 and self.state_timer > 0.3) or self.state_timer > 4.0:
                self.prev_theta = 0.0
                self.scan_history.clear()
                self.set_state('JELAJAH')

        self.drive(v, w)

        if self.ros is not None and self.state != 'JELAJAH':
            self.chosen = (0.0, self.corridor_free(scan, 0.0, hw))
        if self.ros is not None:
            self.ros.publish(self.lidar.getRangeImage(), self.odom_x, self.odom_y, self.odom_yaw,
                             v, w, self.state, self.chosen[0], self.chosen[1], hw)

        # telemetri ke console Webots
        self.log_timer += self.dt
        if self.log_timer >= LOG_PERIOD:
            self.log_timer = 0.0
            print(f"[t={self.time:6.1f}s] state={self.state:7s} "
                  f"depan_bebas={front_free:4.2f}m min_depan={front_min:4.2f}m "
                  f"us_min={us_min:4.2f}m v={v:5.2f}m/s w={w:5.2f}rad/s "
                  f"enc=({self.enc_l.getValue():6.1f},{self.enc_r.getValue():6.1f})",
                  flush=True)

    def run(self):
        print(f"Robot B401 Obstacle Avoidance siap. Lidar: {self.n_rays} sinar, "
              f"FOV {math.degrees(self.fov):.0f} derajat, jangkauan {self.max_range} m", flush=True)
        while self.robot.step(self.dt_ms) != -1:
            self.time += self.dt
            self.step()


if __name__ == '__main__':
    ObstacleAvoider().run()
```